

Prioridades · 29 de agosto de 2026

Este documento cruza tres cosas: la **propuesta de desarrollo** ([DevUP-Propuesta-de-Desarrollo.pdf](#)), el **plan de lo que falta** ([plan-lo-que-falta.md](#)) y **lo que hoy hay de verdad en el repositorio**, comprobado a mano. Donde el documento y la máquina no coincidían, manda la máquina y queda anotado.

1. Tres cosas que los documentos daban por buenas y no lo eran

Los respaldos de la base llevaban dos días sin escribirse. El traspaso del 27 decía «restauración probada, las diez tablas coinciden», y era cierto el día que se probó a mano. Lo que fallaba era el planificador: al reiniciarse el demonio de Docker, `restart: unless-stopped` devuelve todos los contenedores a la vez saltándose el orden de `depends_on`, el respaldo salía antes que Postgres y el bucle se dormía veinticuatro horas. Tres reinicios de Docker, tres días sin copia. **Arreglado y verificado hoy**; el detalle está en [RESPALDOS.md](#).

La lección para el resto del plan: *tener el script* y *tener la copia* son cosas distintas, y solo la segunda cuenta.

La búsqueda global de la rama de Juan Medina ya no hace falta. El plan decía «fusionar `claude/inicio-desarrollo-nu1ftu`». Esa rama salió de una base vieja y trae `0007_búsqueda.sql` y `0008_menciones_y_acceso.sql`, dos números que aquí ya están ocupados por el mundo. Y la búsqueda **ya existe en esta línea**: `0014_global_search.sql`, `routes/search.ts` y la pantalla `/o/[orgId]/buscar`. Fusionar la rama entera sería resolver un conflicto de migraciones para traer una segunda implementación de algo que funciona.

Lo que sí vale de esa rama, y mucho, es **la suite de Playwright**: seis ficheros de prueba, unas 1.100 líneas, cubriendo conversación, voz, cuenta, espacios y búsqueda. Eso es el bloque C entero ya escrito. La tarea correcta no es fusionar, es **cosechar**: llevarse `tests/e2e/`, `playwright.config.ts` y `tsconfig.e2e.json`, y dejar las migraciones donde están.

El despliegue automático fallaría en el primer intento aunque registres el ejecutor. `deploy.yml` hace `git merge --ff-only` sobre esta misma copia de trabajo, que ahora mismo está en una rama de trabajo y tres commits por delante de la principal. El avance en línea recta no existe y el paso falla. O la copia de producción se deja siempre en la rama principal y limpia, o el despliegue clona aparte. Es una decisión de una frase, pero hay que tomarla antes de registrar el ejecutor, no después.

2. Hoy · lo que bloquea y solo puedes hacer tú

	QUÉ	CUÁNTO CUESTA
1	Dónde van los respaldos. Hoy en el mismo disco que la base. Ya protegen de un borrado por error; no protegen de que el disco falle. Una línea <code>RUTA_RESPALDOS=</code> en <code>.env.production</code> apuntando a un disco externo o una unidad de red	2 minutos
2	Registrar el ejecutor (<code>config.cmd</code> + token), después de decidir lo de la copia de trabajo	10 minutos
3	Pedir la extensión de cuota de Spotify y añadir a los compañeros en las 5 plazas del modo desarrollo	15 minutos

Los tres son tuyos porque los tres son credenciales o decisiones, no código.

3. Esta semana · que nada se pierda

Es el bloque A de la propuesta, y no añade una sola función. Con usuarios reales dentro, es lo único que separa «se cayó un rato» de «se perdió».

Al ponerse con ello, los tres puntos resultaron ser otra cosa de la que decía el plan. Ninguno era «escribir la función que falta».

SMTP · hecho lo que era código

El mailer llevaba escrito desde el principio y `nodemailer` es dependencia: no faltaba código, faltaba **poder comprobarlo**. Sin `SMTP_URL` el mailer escribe el mensaje en el registro, que es cómodo y no prueba nada — esa rama no se autentica, no negocia TLS y no descubre que el remitente no está autorizado. Poner credenciales de verdad y esperar a la primera invitación era estrenar todo eso en producción.

Ahora hay un buzón de mentira en el compose de desarrollo (Mailpit, que habla SMTP de verdad y enseña lo recibido en el 8025) y `npm run correo:probar`, que separa las dos mitades: `verify()` prueba conexión y autenticación, y solo después se envía. Un fallo dice si no se llegó al servidor, si no te dejaron entrar, o si te dejaron entrar y luego no aceptaron el mensaje.

Lo que queda es tuyo: dar de alta un proveedor y pegar `SMTP_URL`. Se comprueba en diez segundos.

La bóveda · hecho, y quita una restricción permanente

`RESPALDOS.md` decía que rotar `VAULT_MASTER_KEY` «no puede hacerlo un script» y que, mientras no lo hubiera, la clave no se cambia nunca. Eso convertía una credencial normal en una que no se puede rotar **aunque se filtre**, que es la peor propiedad que puede tener una clave maestra.

Ya existe `npm run boveda:rotar`, con ensayo que no escribe nada, y probado provocando cada fallo — incluida la carrera con la API, con 3.000 filas de relleno y una conexión insertada a mitad.

Lo que queda es tuyo, y no lo puede hacer un script: llevar `.env.production` fuera de esta máquina, a un gestor de contraseñas, y anotar la fecha. Sigue existiendo en un solo archivo, en un solo ordenador, y en ninguna copia.

TURN · deja de ser una decisión

No hay nada que construir: `routes/ice.ts` ya funciona de las dos maneras —Metered.ca si hay clave, y credenciales HMAC para un coturn propio si no—.

Y la rama de auto-alojarlo está cerrada por cómo se despliega hoy: `docker-compose.prod.yml` **no publica ni un solo puerto**. Todo entra por el túnel de Cloudflare, que es un proxy de HTTP y no sabe llevar UDP 3478 a ningún sitio, sobre una NAT doméstica sin IP pública. Un coturn levantado aquí no lo alcanzaría nadie desde fuera.

Así que es un alta, no una decisión: dos variables, `METERED_APP_NAME` y `METERED_API_KEY`. Auto-alojarlo vuelve a la mesa con la decisión 0003, no antes.

4. Después · cerrar lo que está abierto

- **Cosechar las pruebas de navegador** de la rama de Juan Medina (ver §1). Es el bloque C prácticamente hecho, y es justo el tipo de prueba que habría cazado que el almacén de archivos nunca se creaba.
- **Explicar los 404 del conector de GitHub** en la interfaz. Casi nunca es un fallo nuestro, pero lo parece.
- **Cerrar las decisiones 0003 y 0004.** La 0003 —arquitectura de despliegue— ha dejado de ser teórica, y por un motivo más simple del que parecía: Docker Desktop no se cae, **se para al cerrar la aplicación**, y con `AutoStart: false` tampoco vuelve sola al iniciar sesión. Producción vive dentro de una aplicación de escritorio atada a una sesión de usuario. Eso se puede paliar hoy (§4.1) pero no se arregla del todo, porque una herramienta de desarrollo no es un gestor de servicios. La pregunta ya no es si conviene un servidor de verdad, es cuándo.

4.1 Que esté encendido a ratos es una decisión, no un descuido

`AutoStart` está en `false` y **se queda así**. DevUP es un MVP y todavía no tiene usuarios de fuera: tener producción encendida en un portátil las veinticuatro horas cuesta batería, memoria y ruido a cambio de una disponibilidad que hoy no le sirve a nadie. Se abre Docker Desktop cuando se va a trabajar y se cierra al terminar.

Dos consecuencias que conviene tener claras mientras dure:

- **Los contenedores vuelven solos.** Están todos en `restart: unless-stopped`, así que al abrir la aplicación se levantan sin ningún `up -d`. Lo único que se rompía en cada uno de esos ciclos era el respaldo, y eso ya está arreglado.
- **Choca con el despliegue automático**, y conviene saberlo antes de registrar el ejecutor: si alguien empuja a la rama principal con la máquina apagada o con Docker cerrado, el trabajo de despliegue falla en el `docker compose up`. No rompe nada —producción se queda como estaba— pero el aviso en rojo llega igual, y hay que saber leerlo como «no estaba encendido» y no como «el despliegue está roto». La forma limpia de convivir con las dos cosas es desplegar a mano mientras dure el MVP, o aceptar los rojos.
- **El día que haya alguien de fuera dentro, esto deja de valer.** Y entonces la respuesta no es la casilla de arranque automático: es la decisión 0003. Una casilla haría que producción dependiera de que la sesión de Windows esté abierta, que es el mismo problema con otra cara.

5. El trabajo grande, y por qué en este orden

La regla de la propuesta se sostiene: **primero lo que protege, después lo que abarata lo siguiente, y al final lo que solo hace falta cuando duela.**

Primero la interfaz (I1 → I4), y no las pantallas nuevas

Los números del diagnóstico siguen siendo los de hoy, recontados:

MEDIDA	HOY
Botones crudos vs. la primitiva	56 vs. 97
Desplegables y áreas de texto escritos a mano	12
Acciones irreversibles en el cuadro gris del sistema	8
Llamadas a la API sueltas en pantallas	88, con 74 efectos
Armazón de organización	no existe

MEDIDA	HOY
La pantalla de ventas	1.273 líneas

El motivo del orden es de coste, no de gusto: la pantalla grande que falta —la vista de infraestructura — si se construye hoy nace con su séptima cabecera copiada, su propia carga, su propio desplegable y su propio aviso de error. Y habrá que migrarla igual, con la diferencia de que entonces estará en producción.

De los cinco puntos, el que más devuelve por lo que cuesta es **el diálogo de confirmación**: sustituye los ocho cuadros del sistema operativo, que hoy preguntan si borras un cliente sin decir el nombre del producto ni marcar el peligro.

Y luego, en este orden

S7 · vista unificada de infraestructura — cierra la tercera promesa del producto y enciende los muebles que ya están puestos en DevVerse y no hacen nada.

S8 · base de datos como código — el criterio ya existe aquí y se aprendió a base de un fallo silencioso; ese criterio es el producto.

S10 · agentes — va después de S7 a propósito: conviene que la bóveda haya pasado por algo más exigente que dos conectores antes de darle credenciales a un agente. La regla que no se negocia: **el agente propone, la persona aprueba**.

Beta y DevUP ID — depende del bloque A entero. Abrir a gente de fuera un sistema sin correo real y sin copias comprobadas no es una beta.

6. Lo que falta decidir

De las cinco decisiones abiertas de la propuesta, **una ya está resuelta**: el tema claro entró, y entró junto al oscuro con conmutador y respeto por la preferencia del sistema. Quedan cuatro, más una nueva:

1. Dónde van los respaldos. *Bloquea el bloque A entero.*
2. Si el servidor de retransmisión de voz se levanta o se contrata.
3. Los tres ritos que acuñan moneda al empezar, y el techo semanal por persona. Media hora de conversación y define la economía entera de DevVerse.
4. El tamaño definitivo del avatar y cuántos cuerpos base hay de salida.
5. **Nueva**: si la copia de producción se queda siempre en la rama principal y limpia, o si el despliegue clona aparte. Va antes de registrar el ejecutor.

7. Comprobar que sigue en pie

```
npm run typecheck && npm run test:rls && npm run test:world  
npm run respaldo:probar  
docker logs devup-prod-respaldo-base-de-datos-1 --tail 5
```

El último es nuevo y conviene mirarlo de vez en cuando: si aparecen varios `.parcial-*.dump` de cero bytes seguidos y ningún `devup-*.dump` entre ellos, el respaldo está fallando aunque el contenedor se vea levantado.